

Production-Ready 3-Tier Infrastructure Deployment Using Terraform Modules

This project deploys a complete 3-tier web application on AWS using Terraform modules. A user opens a registration form in the browser, submits their details, and the data gets stored in a MySQL database on Amazon RDS. Every layer of the infrastructure is built and managed through Terraform.

Architecture Overview

```
Internet
|
Internet Gateway
|
Public Subnet
|-- Web Server (EC2 + NGINX)
|-- Bastion Host (manual, SSH access only)
|
Private Subnet
|-- App Server (EC2 + Flask) <-- reached via NAT Gateway
|
Database Subnet
|-- RDS MySQL (no internet access)
```

The three tiers are:

- **Web Tier** — NGINX on EC2 in a public subnet. Accepts HTTP requests from users and forwards them to the App Server.
- **Application Tier** — Flask on EC2 in a private subnet. Processes form data and writes it to the database.
- **Database Tier** — RDS MySQL in a private database subnet. Stores user records and is never directly exposed to the internet.

Project Structure

```
.
├── html/
│   └── index.html           # Frontend registration form served by NGINX
├── modules/
│   ├── vpc/                 # VPC, subnets, IGW, NAT Gateway, route tables
│   ├── web/                 # Web Server EC2 instance and security group
│   ├── app/                 # App Server EC2 instance and security group
│   └── rds/                 # RDS instance, DB subnet group, security group
├── scripts/                 # Shell scripts to configure web and app servers
├── main.tf                  # Root config that calls all four modules
└── provider.tf              # AWS provider configuration
```

```
|— variables.tf          # All input variable declarations
|— terraform.tfvars      # Your actual values (region, IPs, passwords,
AMI)
|— outputs.tf            # Prints Web Server IP and RDS endpoint after
apply
|— keypair.tf            # Creates the SSH key pair for EC2 access
|— terraform-key         # Private key file – never commit this to GitHub
|— terraform-key.pub     # Public key file uploaded to AWS
```

Module Explanation

This project is split into four focused Terraform modules. Each module is responsible for one layer of the infrastructure and is completely independent of the others except through the values passed between them in `main.tf`.

VPC Module — `modules/vpc/`

This is the foundation of the entire project. It builds the private network that all other resources live inside.

What it creates:

- A VPC with a /16 CIDR block and DNS support enabled
- Two public subnets for the Web Server and Bastion Host
- Two private subnets for the App Server
- Two database subnets for RDS
- An Internet Gateway so the public subnets can reach the internet
- A NAT Gateway in the public subnet so the App Server in the private subnet can make outbound calls (for installing packages) without being directly reachable from the internet
- A public route table that sends outbound traffic to the Internet Gateway
- A private route table that sends outbound traffic to the NAT Gateway
- Route table associations connecting each subnet to the correct route table

Files inside this module: `main.tf`, `igw.tf`, `nat.tf`, `subnets.tf`, `route_tables.tf`, `variables.tf`, `outputs.tf`

Web Module — `modules/web/`

This module creates the entry point of the application — the server that users connect to directly.

What it creates:

- An EC2 instance in the public subnet running Amazon Linux
- A security group attached to that instance
- The security group allows inbound HTTP on port 80 from anywhere so users can reach the form
- The security group allows inbound SSH on port 22 only from your specific IP address
- All outbound traffic is allowed so NGINX can forward requests to the App Server

Files inside this module: `ec2.tf`, `security_group.tf`, `variables.tf`, `outputs.tf`

App Module — `modules/app/`

This module creates the backend server that lives in the private subnet and handles all the application logic.

What it creates:

- An EC2 instance in the private subnet running Amazon Linux
- A security group attached to that instance
- The security group allows inbound traffic on port 5000 only from the Web Server's security group — not from the internet
- The security group allows SSH only from the Bastion Host's security group
- All outbound traffic is allowed so Flask can reach the RDS database

Files inside this module: `ec2.tf`, `security_group.tf`, `variables.tf`, `outputs.tf`

RDS Module — `modules/rds/`

This module creates the managed MySQL database that stores user data.

What it creates:

- An RDS MySQL instance on a db.t3.micro instance type with General Purpose SSD storage
- A DB subnet group spanning both database subnets across two availability zones
- A security group that allows inbound MySQL traffic on port 3306 only from the App Server's security group
- The RDS instance is set to not be publicly accessible — it can only be reached from inside the VPC

Files inside this module: `rds.tf`, `subnet_group.tf`, `security_group.tf`, `variables.tf`, `outputs.tf`

Deployment Steps

Before You Start

Make sure you have these installed on your machine:

- Terraform
 - AWS CLI
 - VS Code with the Terraform and AWS Toolkit extensions
-

Step 1 — Create an IAM User

Go to AWS Console and create an IAM user named `terraform-user`. Attach the `AdministratorAccess` policy to it.

Then go to that user's Security Credentials tab and create an Access Key. Download the CSV file — you need the Access Key ID and Secret Access Key for the next step.

Step 2 — Configure AWS CLI

Run `aws configure` in your terminal. Enter your Access Key ID, Secret Access Key, preferred region, and `json` as the output format.

Step 3 — Clone the Repository

Clone this repository to your local machine and open the folder in VS Code.

Step 4 — Fill In Your Variable Values

Open `terraform.tfvars` and fill in the required values. This includes your AWS region, your local IP address for SSH access, the database password, and the AMI ID for the EC2 instances.

Do not commit `terraform.tfvars` to GitHub if it contains sensitive values like passwords. It is already listed in `.gitignore`.

Step 5 — Deploy with Terraform

Open a terminal in the project root and run these commands one by one:

Run `terraform init` to download the AWS provider plugin and initialize all four modules.

Run `terraform validate` to check for any syntax errors in your configuration before doing anything on AWS.

Run `terraform plan` to see a complete preview of what Terraform will create. Read through this carefully before going further.

Run `terraform apply` and type `yes` when prompted. Terraform will create all resources across all four modules. This usually takes 5 to 10 minutes.

When it finishes, check the terminal output. The outputs defined in `outputs.tf` will show you the Web Server's public IP address and the RDS endpoint. Save these — you will need them in the next steps.

Step 6 — Create the Bastion Host (Manual)

The Bastion Host is not managed by Terraform in this project. Create it manually from the AWS Console.

Launch an EC2 instance with Amazon Linux 2 in the same public subnet as the Web Server. Give it a security group that only allows SSH on port 22 from your IP address. This server's only purpose is to act as a secure jump box into the private App Server.

Step 7 — Configure the Web Server

SSH into the Web Server using its public IP and the `terraform-key` private key file.

Install NGINX on the server using the setup script in the `scripts/` folder or manually. Configure NGINX to act as a reverse proxy that forwards requests to the `/register` path over to the App Server's private IP on port 5000. Start the NGINX service and enable it so it starts automatically on reboot.

Copy the `index.html` file from the `html/` folder in the repo to the NGINX web root directory so users see the registration form when they visit the Web Server's IP.

Step 8 — Configure the App Server

SSH into the Bastion Host first. Then from the Bastion, SSH into the App Server using its private IP and the same key file.

Install Python 3, pip, Flask, and PyMySQL on the App Server using the setup script from the `scripts/` folder or manually.

Set the following environment variables on the App Server so Flask can connect to RDS: the RDS endpoint as `DB_HOST`, the admin username as `DB_USER`, your database password as `DB_PASSWORD`, and the database name as `DB_NAME`.

Copy the Flask application script from `scripts/` to the App Server and run it in the background using `nohup` so it continues running after you close the terminal.

Step 9 — Verify the Database

From the App Server, install the MySQL client. Connect to the RDS instance using the endpoint from the Terraform outputs along with your admin credentials.

Check that the database and the users table exist. After testing the form from the browser, run a select query on the users table to confirm data is being saved correctly.

Step 10 — Test the Full Application

Open a browser and go to the Web Server's public IP address. The registration form should load. Fill in the form and submit it. You should receive a success message.

Check the RDS database to confirm the record was inserted. If everything is working, the full 3-tier flow is complete.

if Destroying Everything

When you are done and want to remove all AWS resources to stop incurring charges, run `terraform destroy` and type `yes` to confirm.

Remember that the Bastion Host was created manually, so you need to terminate it separately from the AWS Console.

Security Best Practices

This project follows several important security principles that are worth understanding, especially if you plan to extend it or use similar patterns in production.

Each Layer Only Talks to the Layer Directly Below It

The security groups are designed so that no layer has more access than it needs. The internet can only reach the Web Server on port 80. The Web Server can only reach the App Server on port 5000. The App Server can only reach the database on port 3306. The database has no outbound route to the internet at all. This is called the principle of least privilege, and it limits how far an attacker can move if one layer is compromised.

The App Server and Database Are Never Exposed to the Internet

Both the App Server and RDS instance live in private subnets with no direct internet route. The App Server can make outbound requests through the NAT Gateway to install packages, but nothing from the internet can initiate a connection to it. The RDS instance has `publicly_accessible` set to false, meaning it cannot be reached from outside the VPC under any circumstances.

SSH Access Is Restricted to Specific IPs

The Web Server and Bastion Host security groups only allow SSH from your specific IP address defined in `terraform.tfvars`. This means even if someone discovers your server's public IP, they cannot attempt to log in over SSH unless they are coming from your IP.

The Bastion Host Is the Only SSH Entry Point to Private Servers

You never SSH directly into the App Server from your machine. You always go through the Bastion Host first. This means only one machine needs to be hardened and monitored for SSH access, and the private subnet stays isolated.

Private Keys Are Not Stored in the Repository

The `terraform-key` private key file is listed in `.gitignore` so it is never accidentally committed and pushed to GitHub. Anyone who gets access to your repository should not be able to log into your servers.

Database Credentials Are Passed as Variables, Not Hardcoded

The RDS password is defined as a variable in `variables.tf` and set in `terraform.tfvars`, which is excluded from version control. This prevents credentials from being embedded in code that gets committed to a public or shared repository.

NAT Gateway Instead of Making the App Server Public

Rather than assigning a public IP to the App Server so it can download packages, this project uses a NAT Gateway in the public subnet. The App Server can reach the internet for outbound calls, but inbound connections from the internet are still blocked. This is the correct pattern for private subnets that still need internet access.

Subnets Span Multiple Availability Zones

The subnets are spread across two availability zones. For the RDS DB subnet group, this is actually required by AWS. But it also means the infrastructure is more resilient — if one availability zone has an issue, the other is still available.

What I Learned From This Project

- How to break Terraform infrastructure into focused, reusable modules
- How AWS VPC networking works including subnets, route tables, Internet Gateway, and NAT Gateway
- How to design layered security groups so each tier only communicates with what it needs
- How a 3-tier architecture cleanly separates web, application, and database concerns
- How to configure NGINX as a reverse proxy
- How to connect a Flask application to a managed RDS MySQL database using environment variables
- How to securely access private EC2 instances using a Bastion Host
- Why the NAT Gateway pattern is better than making backend servers publicly accessible
- How to verify a full end-to-end deployment by checking logs, database records, and the browser